

Lecture 2:

Bit Operations and UTF-8 (Unicode!)

CSE 29: Systems Programming and Software Tools
Aaron Schulman (Shalev)



Lecture 2 Overview

- ▣ Examples of processing strings in C
- ▣ How computers go beyond English characters
- ▣ How we can print/control individual bits in memory in C

Week 1 Announcements

- **PA1 is available on <https://cse29.site> & PrairieLearn**
 - Read spec on .site, then do PrairieLearn, then do full proj
 - Get started immediately and go to all available office hours
 - Start all parts that you can do right now (ASCII+strings)
 - Due on Monday the 12th
 - Remember you can resubmit to get to full credit if you miss points.

We need your GitHub account for PAs



Fill out this sheet with your GitHub account (e.g., `aschulm` in `github.com/aschulm`)

We are ready for UTF-8 (ASCII++)

- How to handle the thousands of characters used in languages around the world?
 - ◆ ASCII does not define:
 - » Spanish: José
 - » Chinese: 中
 - » Emoji: 🦀
 - » And many more...
- 256 bit patterns (one byte) is not enough to represent all characters!
- Challenge: Millions of lines of code were written that assumed one byte ASCII chars

Solution - UTF-8: Introduced in at a conference in San Diego



Hello World
or
Καλημέρα κόσμε
or
こんにちは 世界

Rob Pike
Ken Thompson

AT&T Bell Laboratories
Murray Hill, New Jersey 07974

ABSTRACT

Plan 9 from Bell Labs has recently been converted from ASCII to an ASCII-compatible variant of Unicode, a 16-bit character set. In this paper we explain the reasons for the change, describe the character set and representation we chose, and present the programming models and software changes that support the new text format. Although we stopped short of full internationalization—for example, system error messages are in Unixese, not Japanese—we believe Plan 9 is the first system to treat the representation of all major languages on a uniform, equal footing throughout all its software.

Introduction

The world is multilingual but most computer systems are based on English and ASCII. The release of Plan 9 [Pike90], a new distributed operating system from Bell Laboratories, seemed a good occasion to correct this chauvinism. It is easier to make such deep changes when building new systems than by refitting old ones.

The ANSI C standard [ANSIC] contains some guidance on the matter of 'wide' and 'multi-byte' characters but falls far short of solving the myriad associated problems. We could find no literature on how to convert a *system* to larger character sets, although some individual *programs* had been converted. This paper reports what we discovered as we explored the problem of representing multilingual text at all levels of an operating system, from the file system and kernel through the applications and up to the window sys-

Created on a napkin at a diner in New Jersey



Demo: UTF-8

Big Idea:

- Single byte characters when inspect()'ing ASCII Characters
- Multi-byte characters when inspect()'ing non-ASCII characters

Printing binary representation of memory



- %u – unsigned integer representation
 - ◆ 'é' is 4294967235 4294967209
- %b – binary representation (4 bytes of bits)
 - ◆ 'é' is 1111111111111111111111111111100001
111111111111111111111111111110101001
- %hhb - single byte (hh is “half half” of 4 = 1)
 - ◆ 'é' is 11000011 10101001
- %hhu – single-byte unsigned char as number
 - ◆ 'é' is 195 169



Negative Integers in Computers

- Integers are stored as binary numbers; binary has no sign (+/-)!
 - ◆ e.g., $1\ 0\ 1\ 0 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 = 10$
- Must encode somehow in the binary digits that a number is neg
- We need a way that is efficient (doesn't waste bits) and simple to implement in the hardware of a CPU

Sign Magnitude – Simple but inefficient



- The simplest way to do this would be to reserve a bit for the sign

<u>S</u>	<u>0</u>	<u>0</u>	
0	0	1	= 1
1	0	1	= -1

- *Inefficient* both in terms of storage and hardware:
 - ◆ Two ways to represent zero (**0** and **-0**)
 - ◆ Math hardware in CPU needs to handle positive and negative differently
 - » Adding a positive number to a negative number needs to read sign bit
 - » Adding positive to positive does not



Two's Complement

- What if we make the MSB equal to -2^{MSB} ?
 - ◆ In other words, if the MSB is set, the number becomes negative with that magnitude

MSB							LSB
-128	64	32	16	8	4	2	1
-2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰

- Minimum will have higher magnitude than the maximum (by -1)
 - ◆ Min (-128)
 - ◆ Max ($127 = 64 + 32 + 16 + 8 + 4 + 2 + 1$)
- Only one zero, and hardware is the same as an unsigned int!

Two's Complement compared to Unsigned



Two's Complement has a lower max compared to unsigned (power of 2)

MSB				LSB			
-128	64	32	16	8	4	2	1
-2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰

Max = 127

128	64	32	16	8	4	2	1
2 ⁷	2 ⁶	2 ⁵	2 ⁴	2 ³	2 ²	2 ¹	2 ⁰

Max = 255

8 bits size (1 byte)



Sign extension in 2's complement

								1	1	1	1	1	1	1	1	= -1
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	= -1

[When copying, extend the sign bit]



Lecture Review:

- Figure out what the 2's complement value is:

1	0	0	1
-2^3	2^2	2^1	2^0

- How many bytes can represent UTF-8 characters?
- What is one way to `printf()` the bits stored in memory?